



Natural Language Processing: Transformers

HSE Faculty of Computer Science

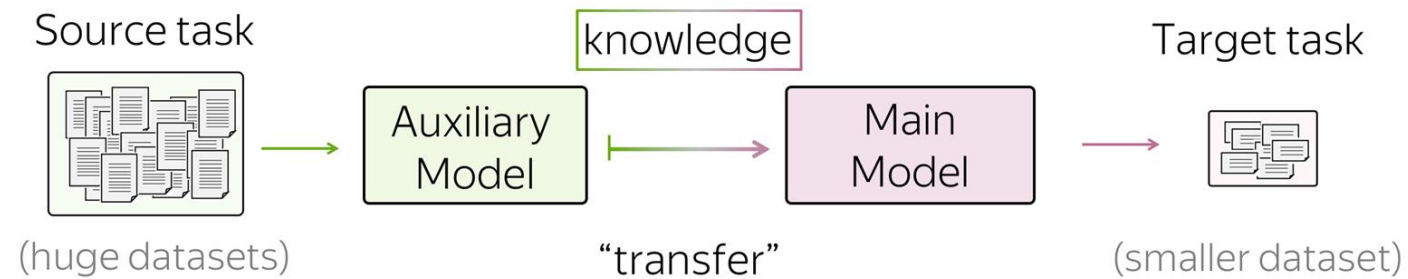
Murat Khazhgeriev



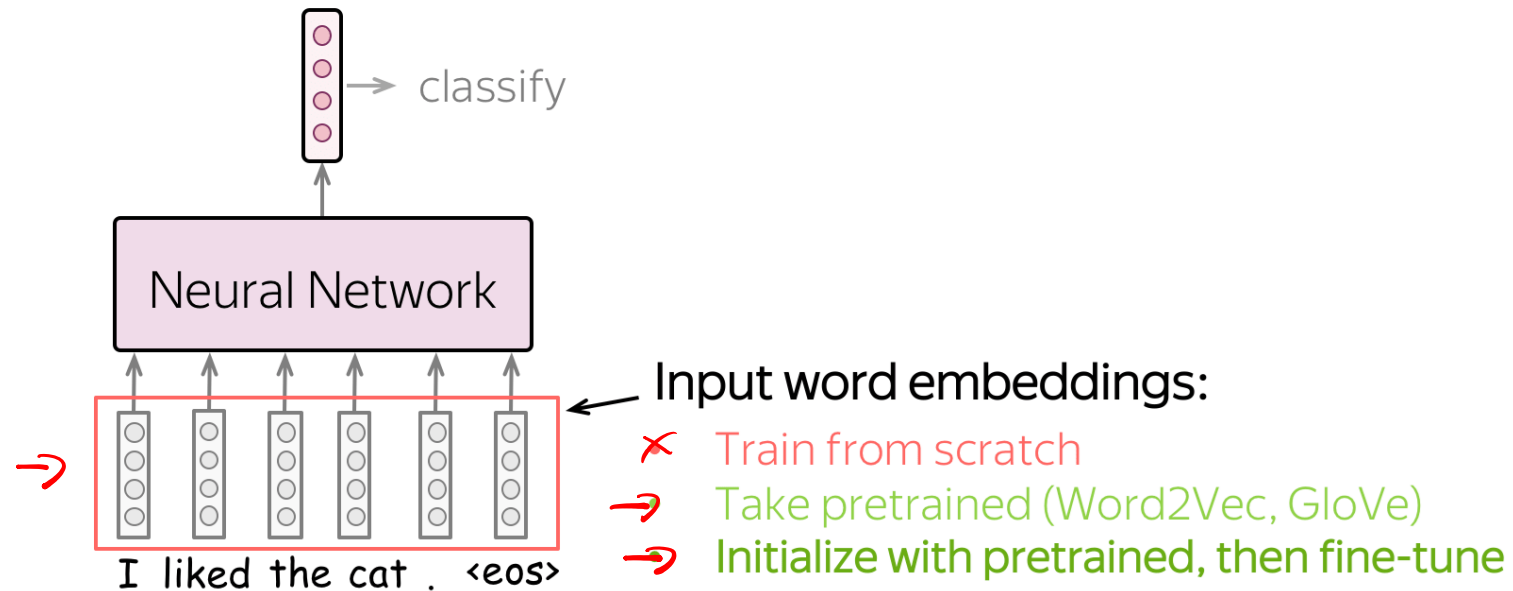
Table of Content

- **The power of transfer learning**
- From word-specific to contextual embeddings
- Transformer architecture

Training a model on a general task can benefit a downstream one



Training a model on a general task can benefit a downstream one



Training a model on a general task can benefit a downstream one

- Train from scratch

What they will know:



May be not enough to learn relationships between words

- Take pretrained (Word2Vec, GloVe)

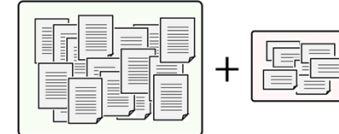
What they will know:



Know relationships between words, but are **not** specific to the task

- Initialize with pretrained, then fine-tune

What they will know:



Know relationships between words and adapted for the task

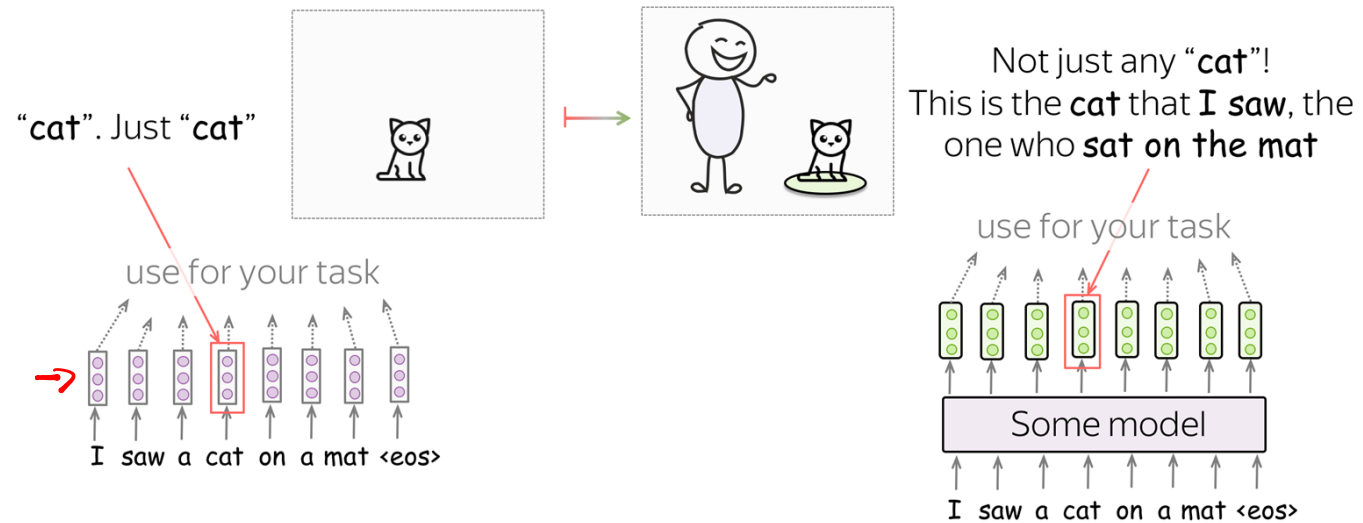
“Transfer” knowledge from a huge unlabeled corpus to your task-specific model



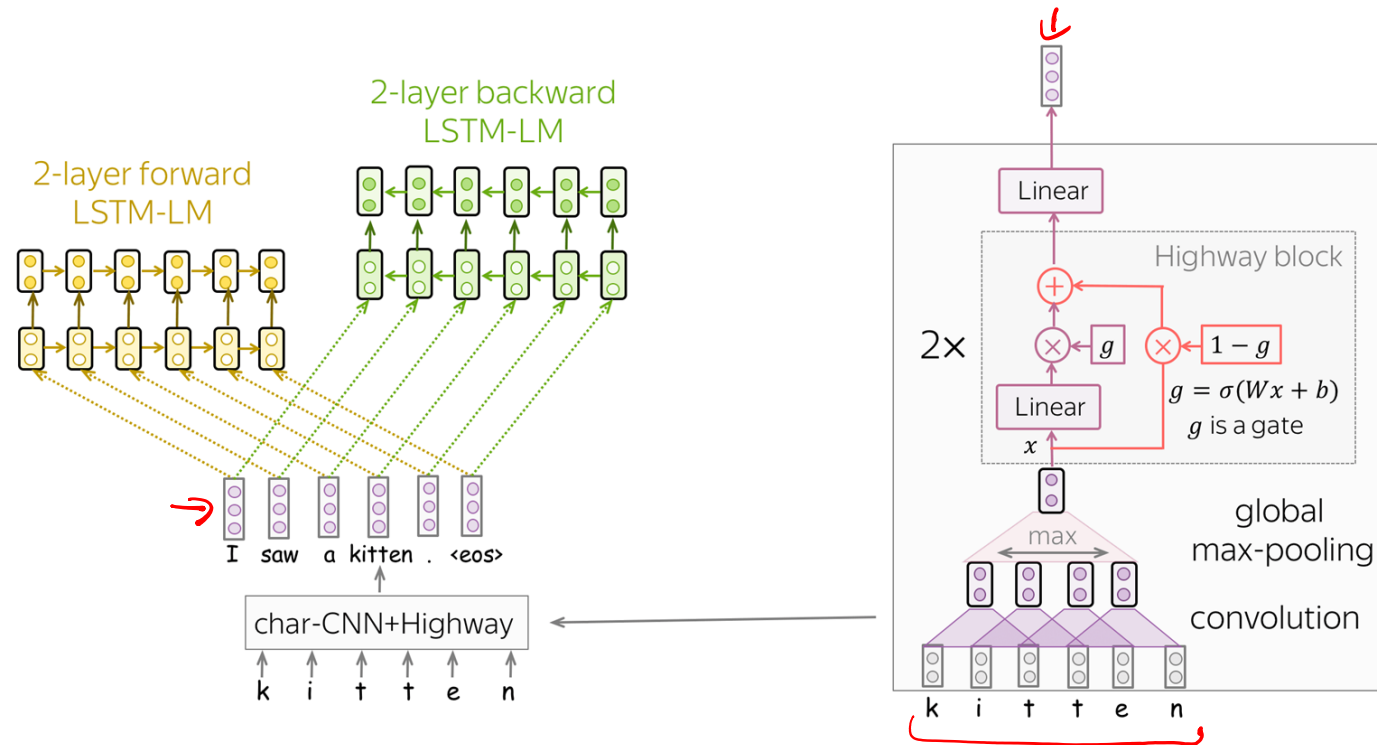
Table of Content

- The power of transfer learning
- **From word-specific to contextual embeddings**
- Transformer architecture

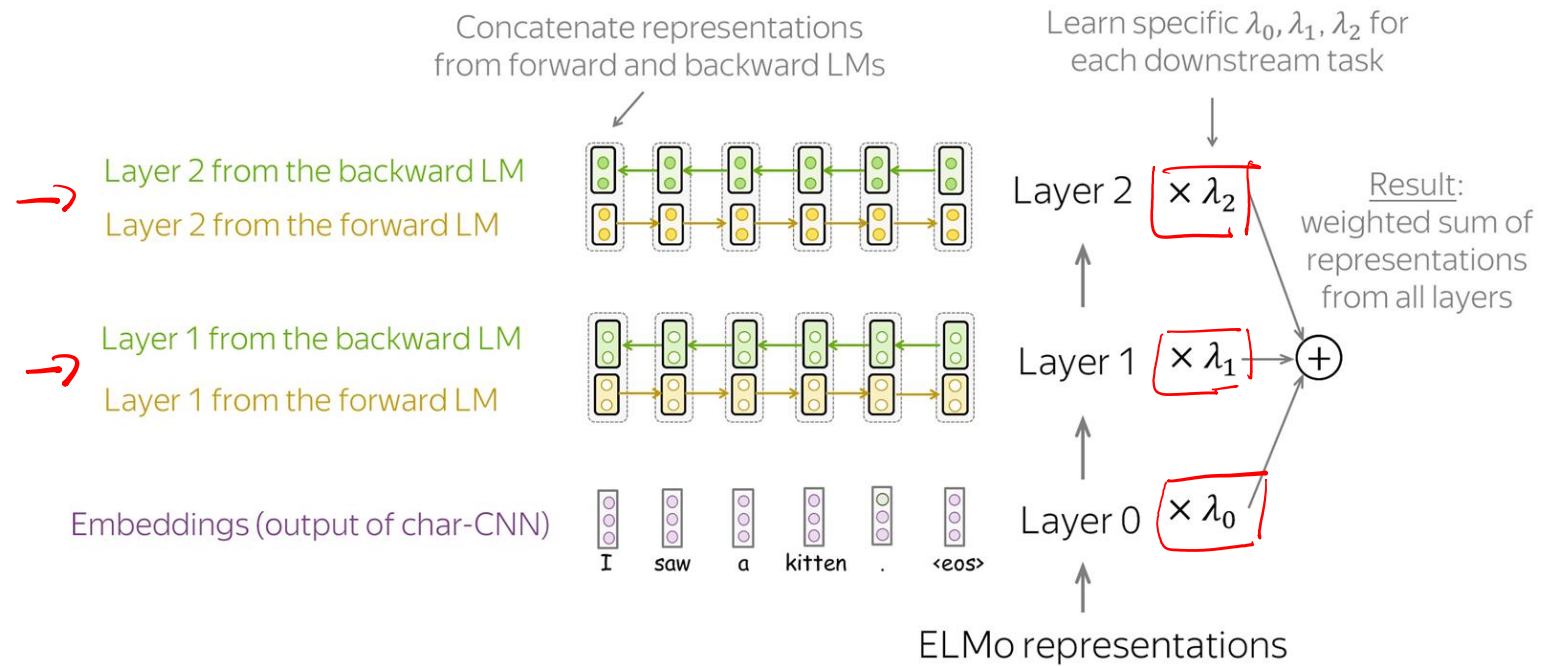
Not just a cat, but the cat!



Train a “translator” from word-specific to “contextual” space



Multiple layers to capture low-level and high-level context



From an embedding generator to a universal model

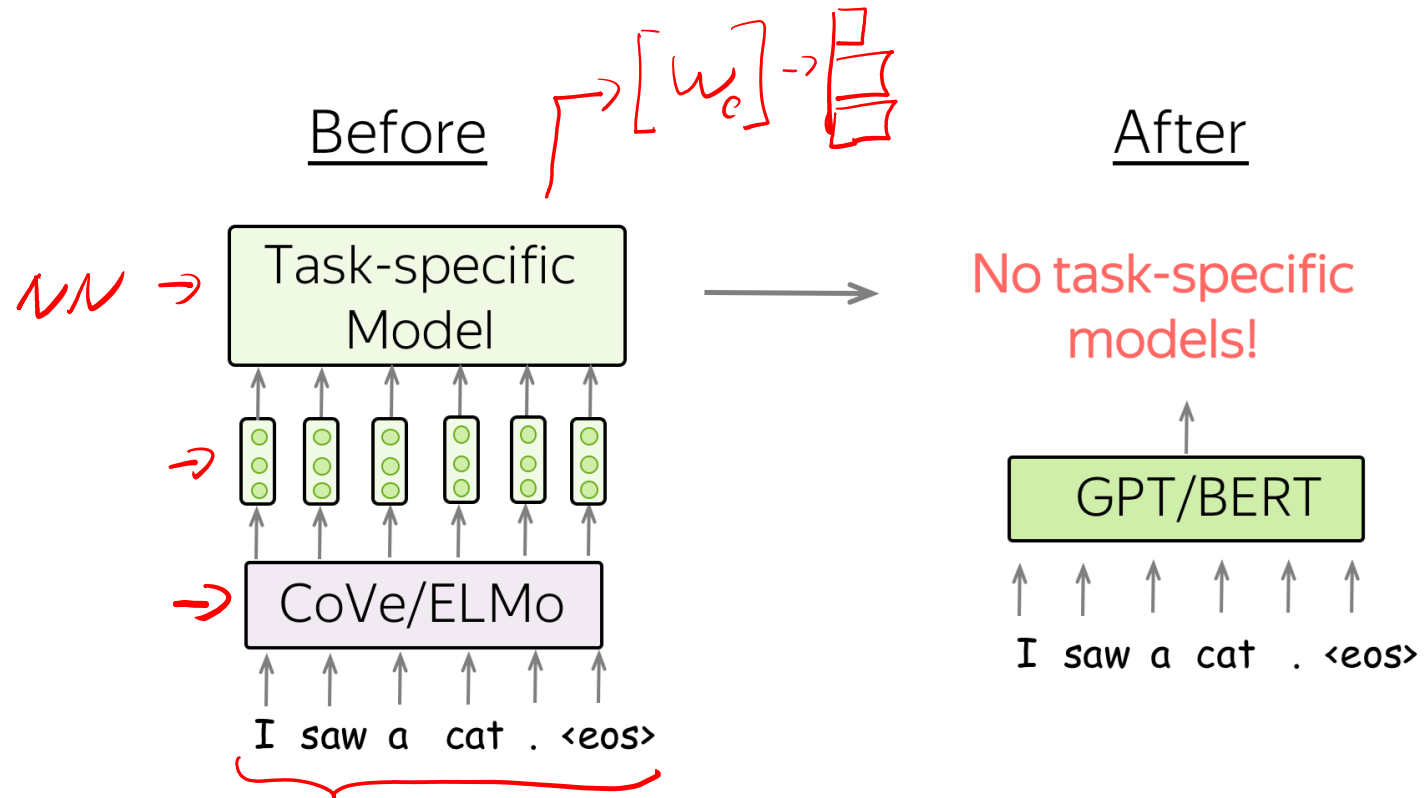
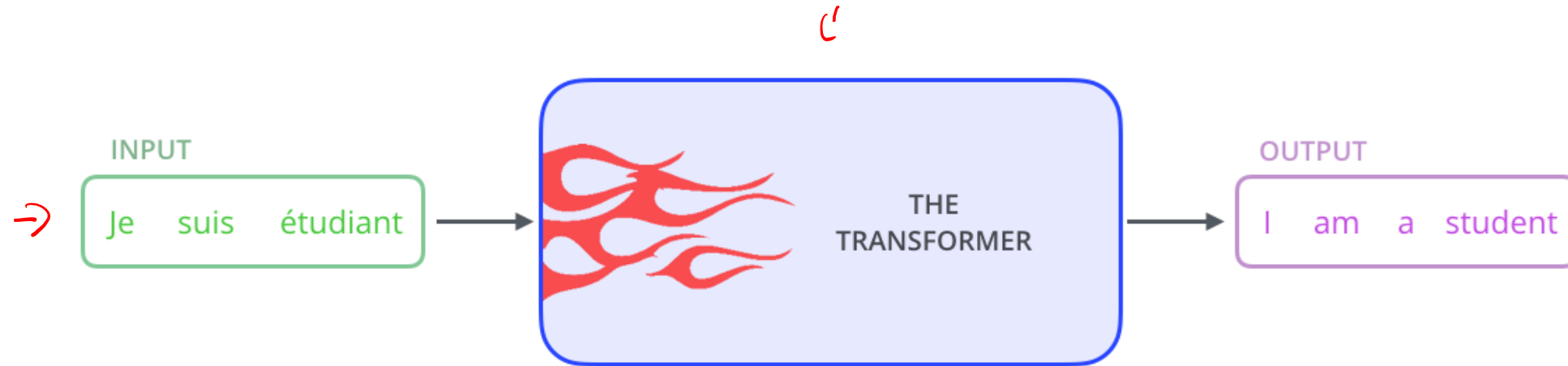




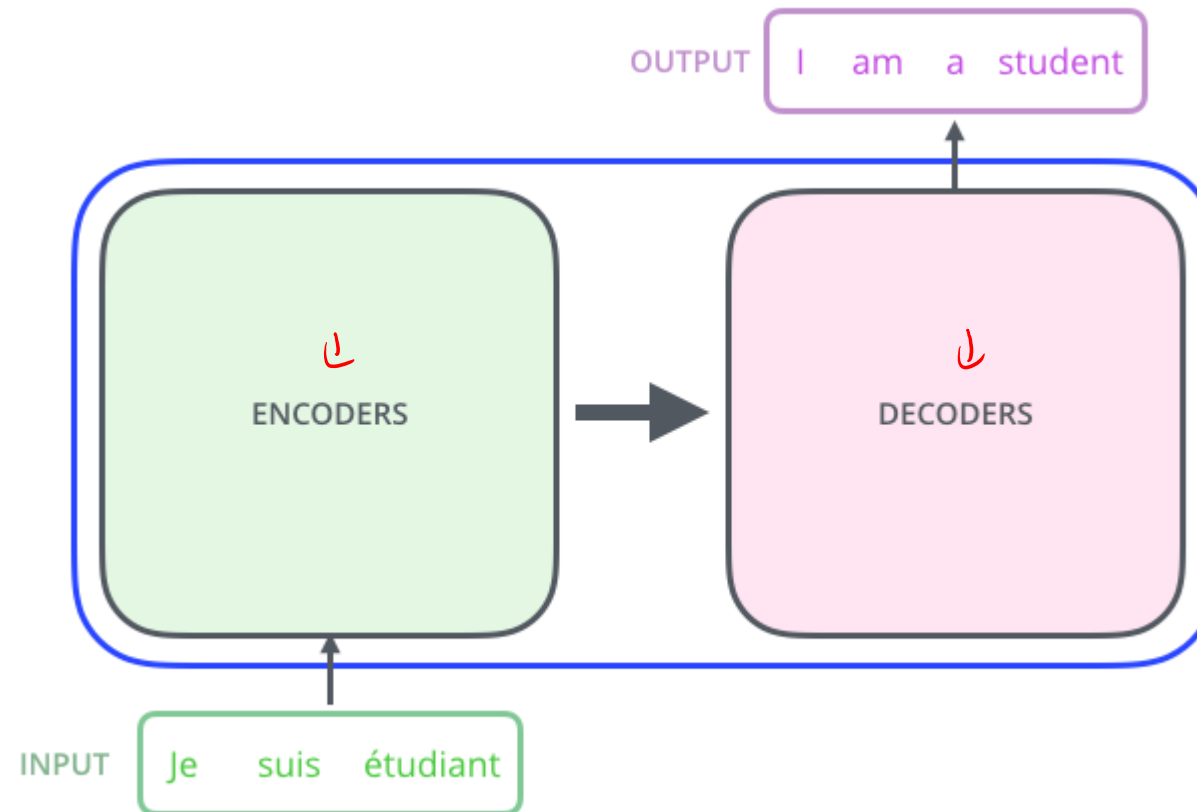
Table of Content

- The power of transfer learning
- From word-specific to contextual embeddings
- **Transformer architecture**

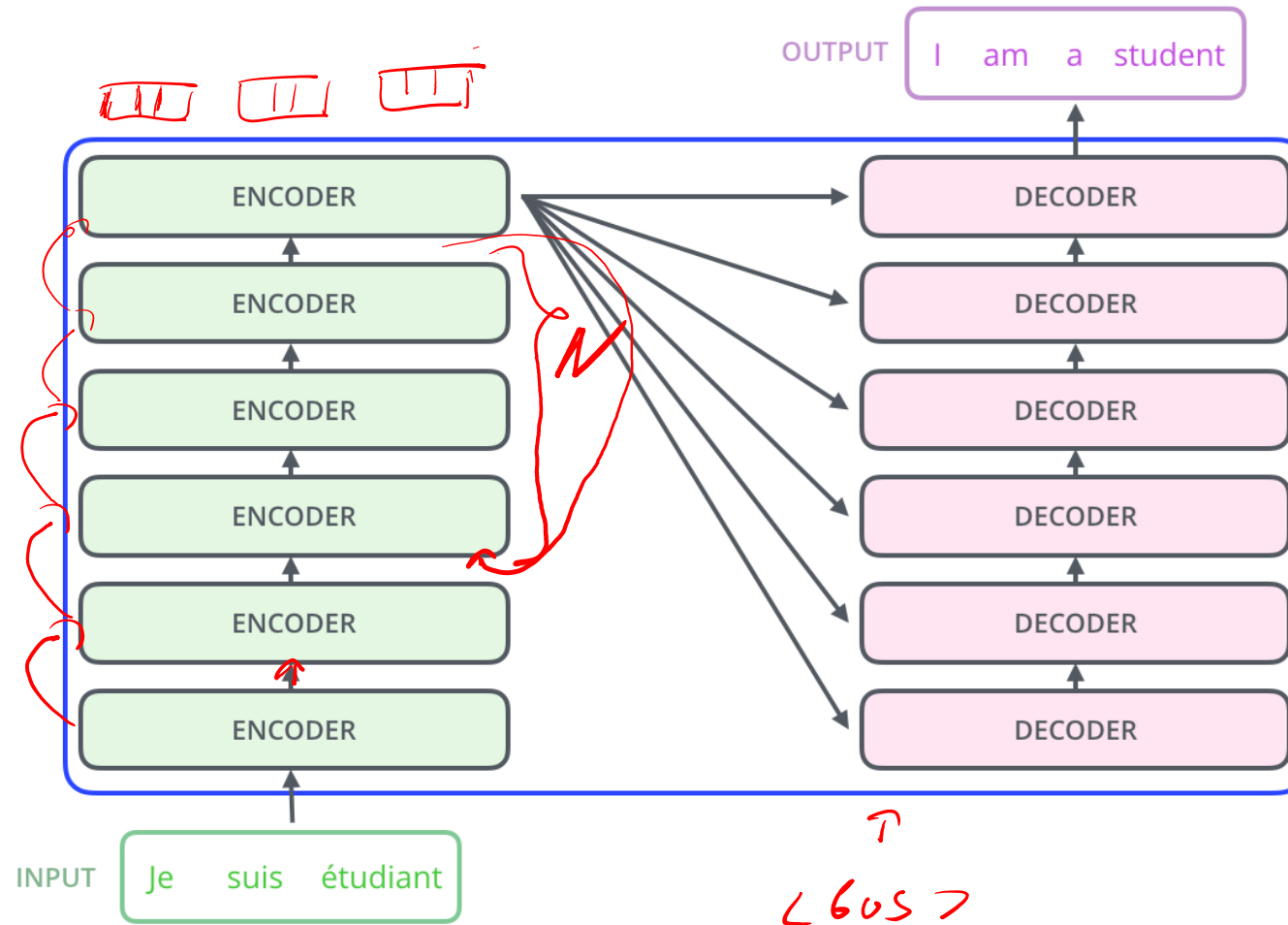
Transformer is an example of Encoder-Decoder architecture



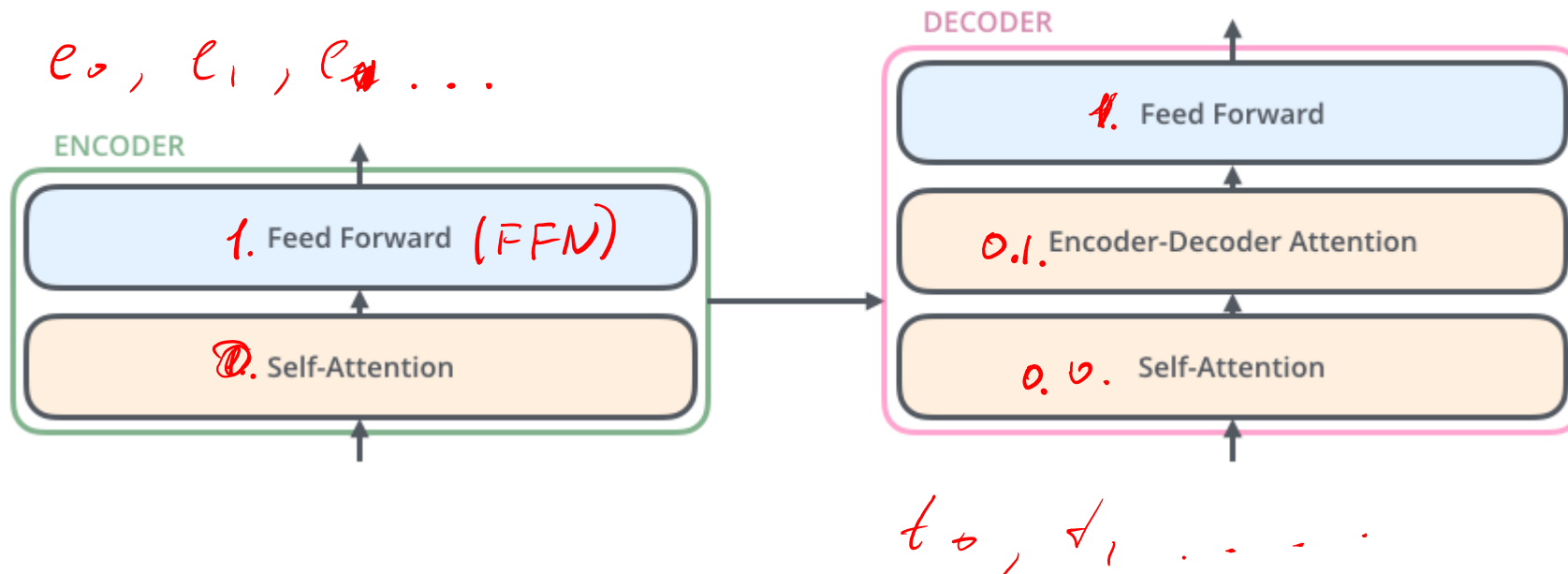
Transformer is an example of Encoder-Decoder architecture



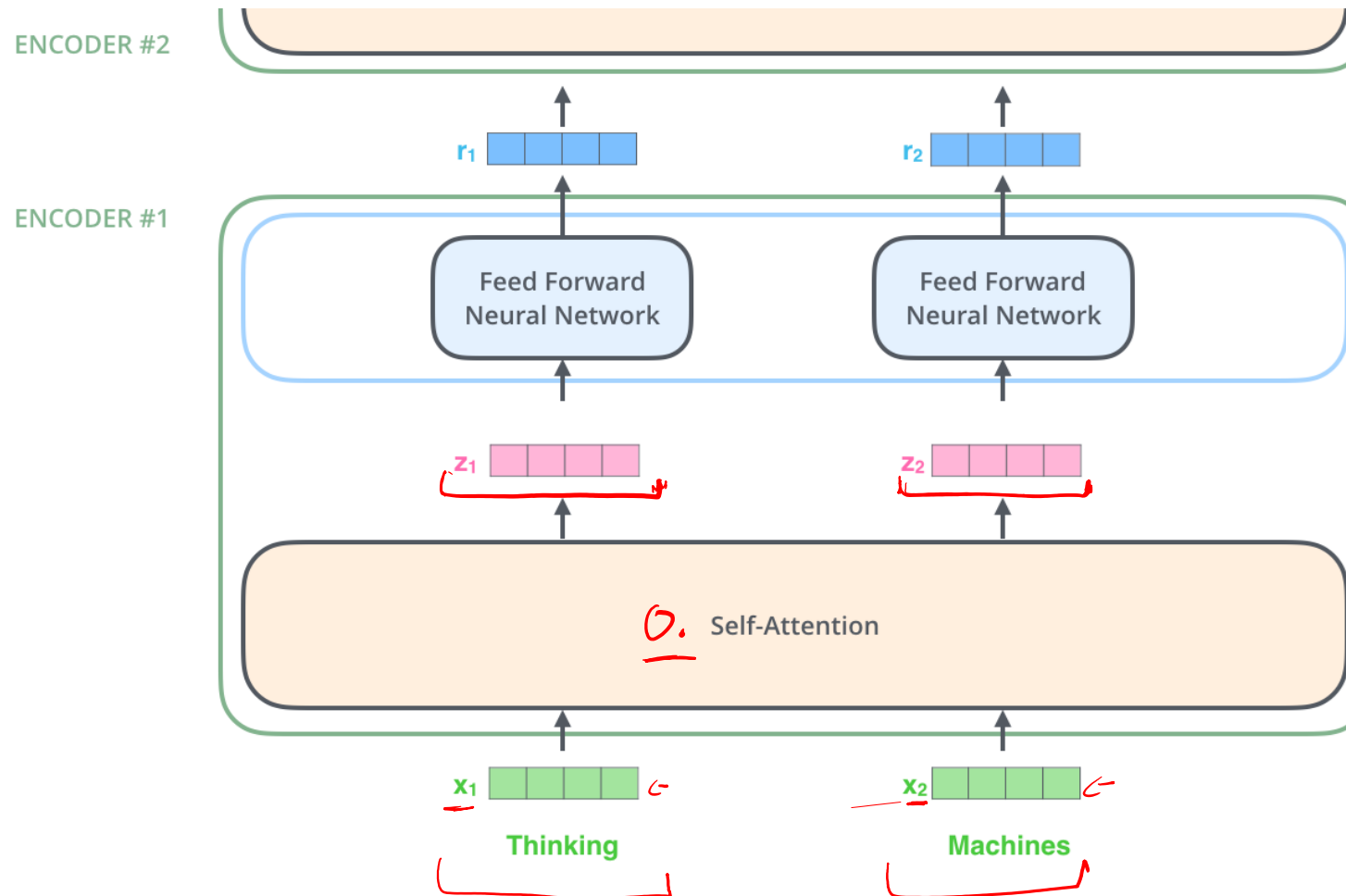
Transformer is an example of Encoder-Decoder architecture



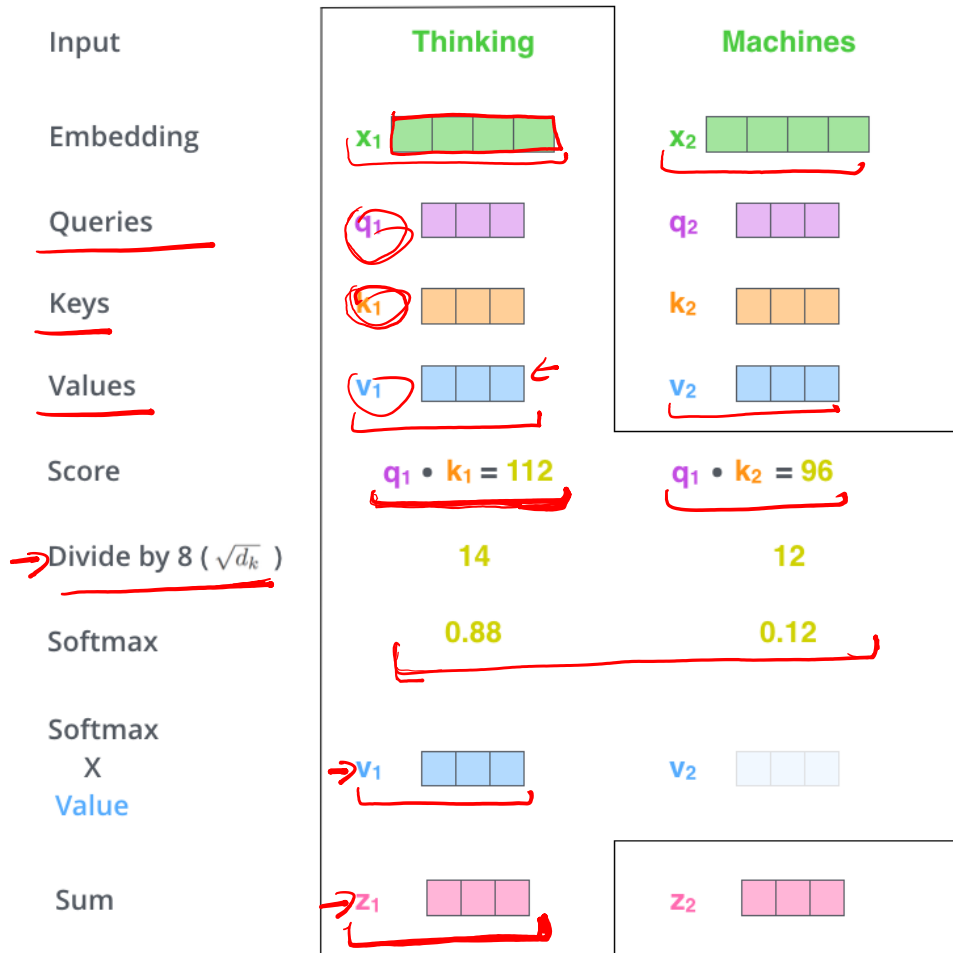
Always two there are. No more, no less. An Attention and an FFN. © Yoda



Always two there are. No more, no less. An Attention and an FFN. © Yoda



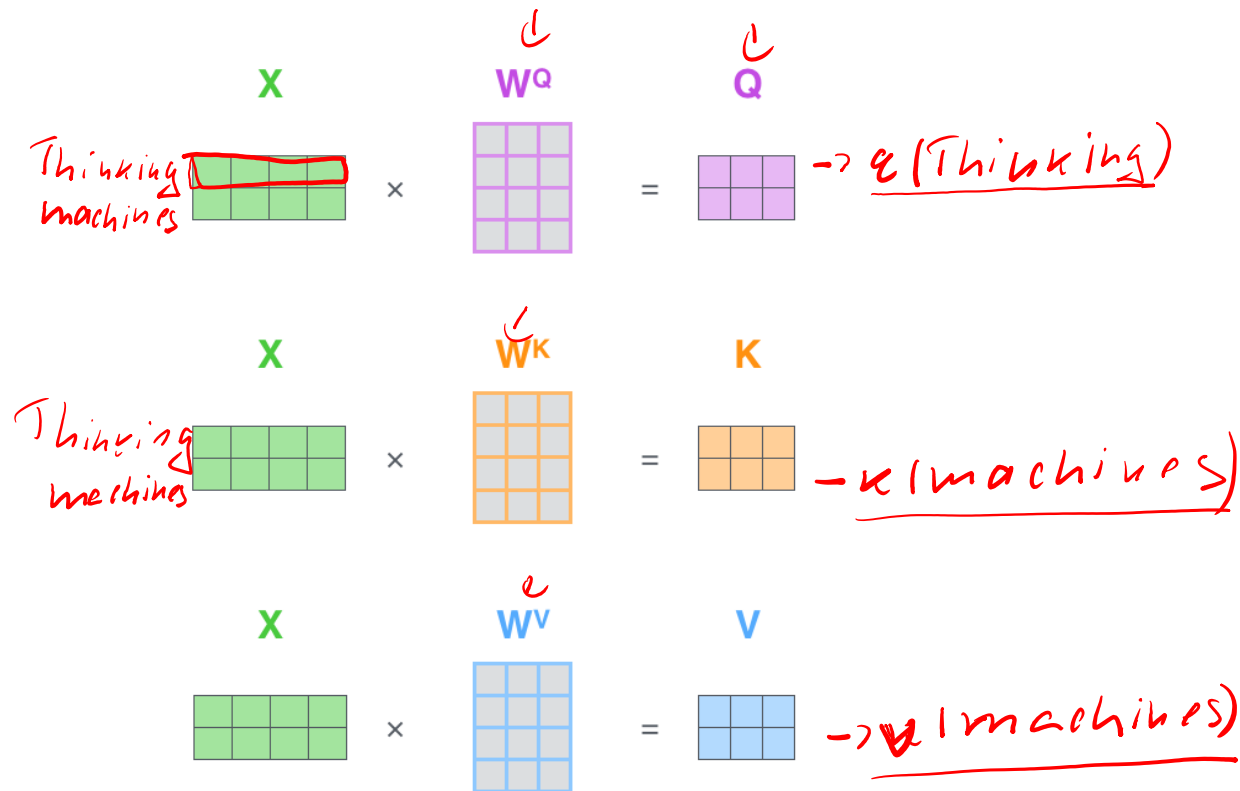
Inside a Self-Attention



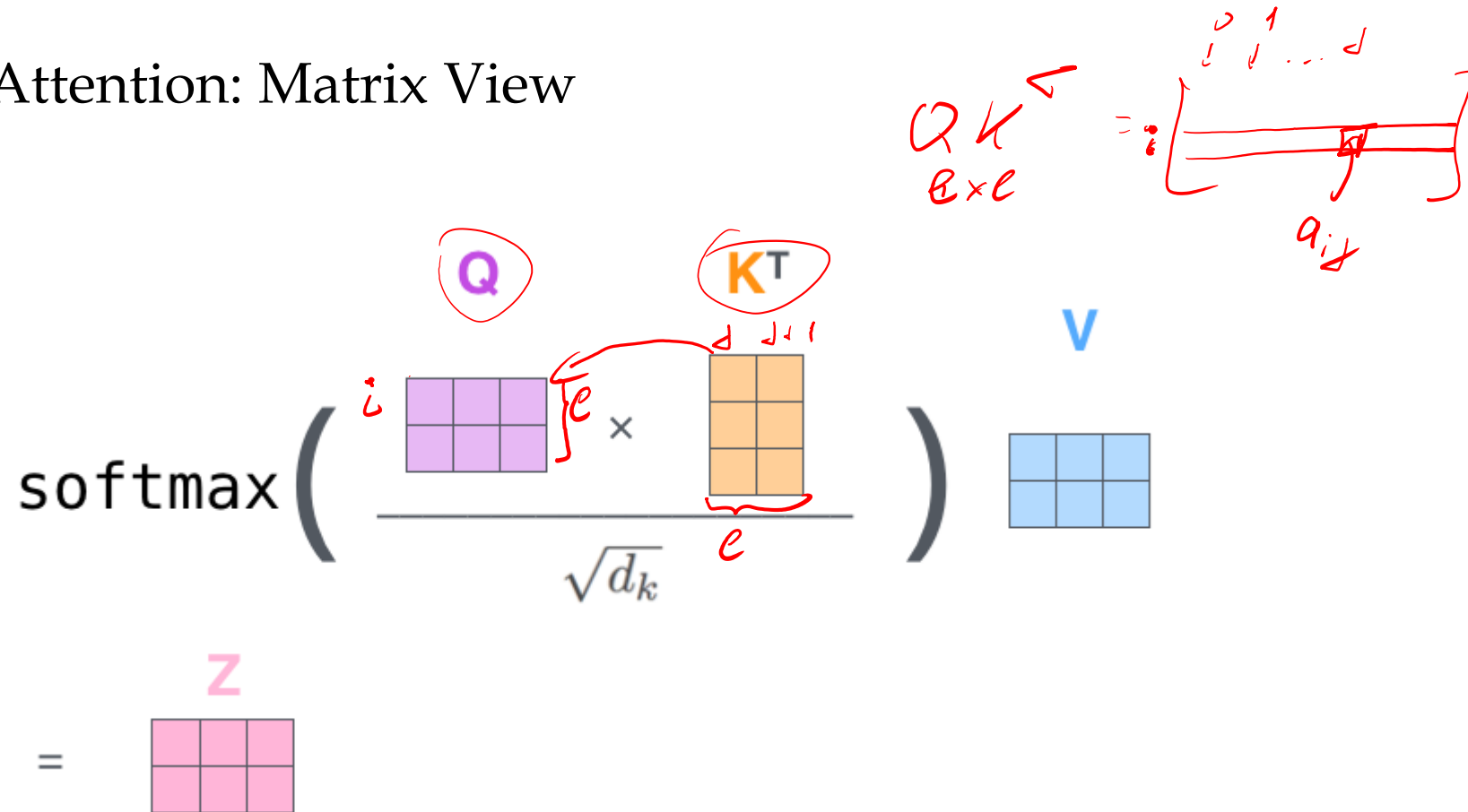
$$\begin{matrix} w_q & \rightarrow & q \\ x_1 & \rightarrow & k \\ w_k & \rightarrow & k \\ w_v & \rightarrow & v \end{matrix}$$

$$\begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix} \xrightarrow{\text{softmax}} \begin{bmatrix} \square \\ \square \\ \square \end{bmatrix}$$

Inside a Self-Attention: Matrix View



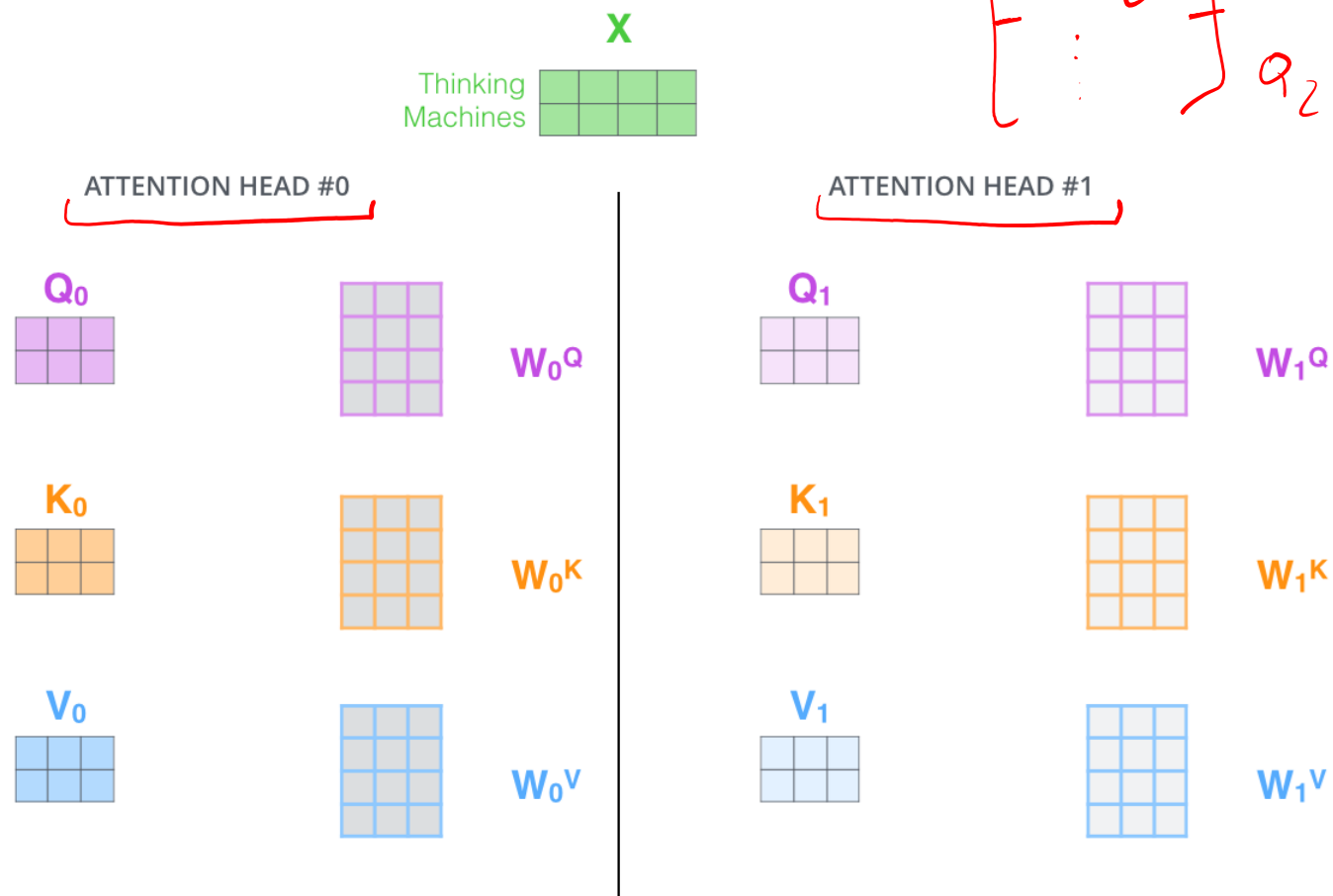
Inside a Self-Attention: Matrix View


$$\text{softmax} \left(\frac{\overset{\text{Q}}{\begin{matrix} \text{ } & \text{ } & \text{ } & \text{ } \\ \text{ } & \text{ } & \text{ } & \text{ } \\ \text{ } & \text{ } & \text{ } & \text{ } \end{matrix}} \times \overset{\text{K}^T}{\begin{matrix} \text{ } & \text{ } & \text{ } \\ \text{ } & \text{ } & \text{ } \\ \text{ } & \text{ } & \text{ } \end{matrix}}}{\sqrt{d_k}} \right) \begin{matrix} \text{V} \\ \begin{matrix} \text{ } & \text{ } & \text{ } \\ \text{ } & \text{ } & \text{ } \end{matrix} \end{matrix}$$
$$= \overset{\text{Z}}{\begin{matrix} \text{ } & \text{ } & \text{ } \\ \text{ } & \text{ } & \text{ } \end{matrix}}$$

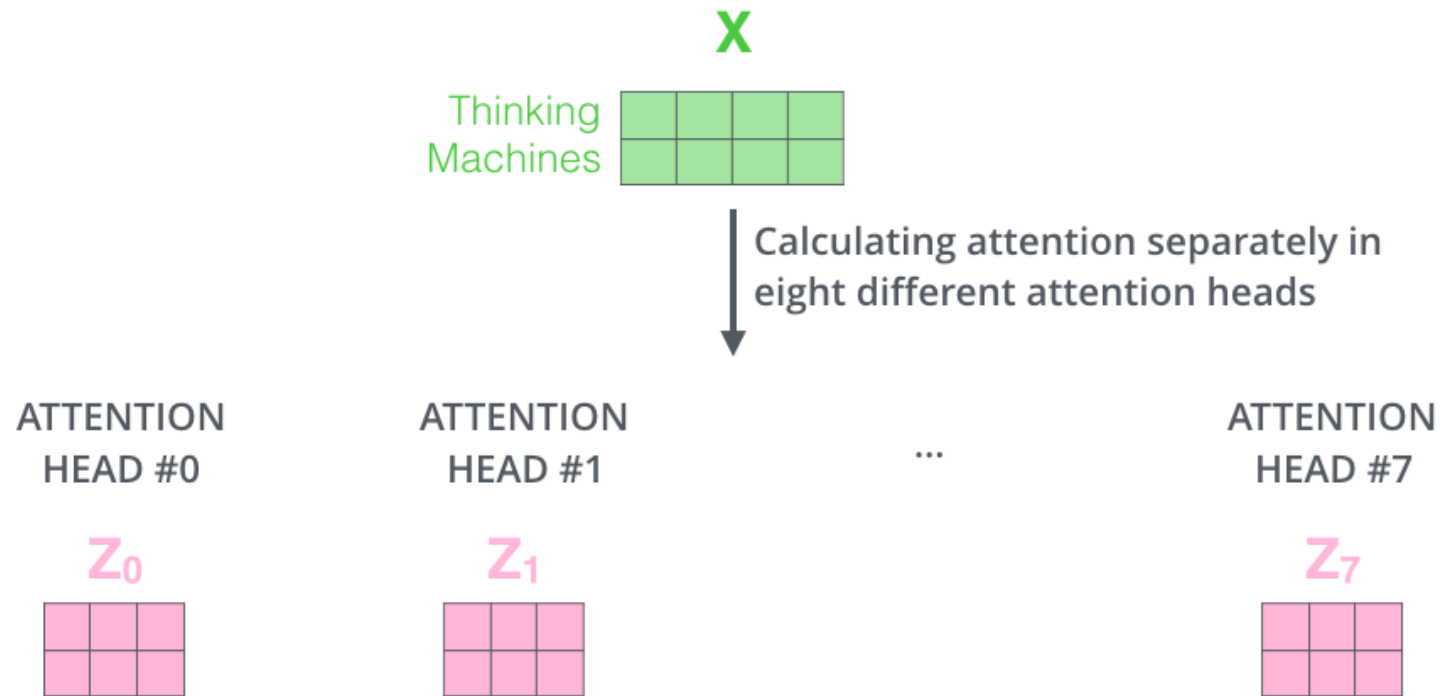
Handwritten red annotations:

- $Q \in \mathbb{R}^{B \times e}$
- $K^T \in \mathbb{R}^{e \times B}$
- Dot product of row i of Q and column j of K^T results in a_{ij} .

A beast with many heads

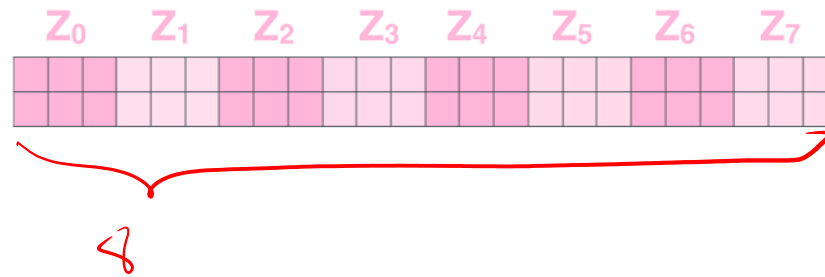


A beast with many heads



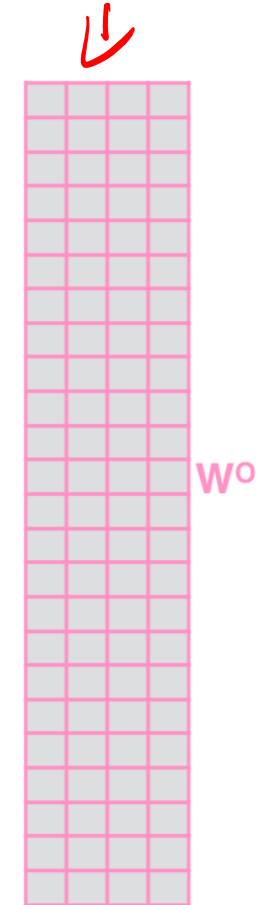
A beast with many heads

1) Concatenate all the attention heads

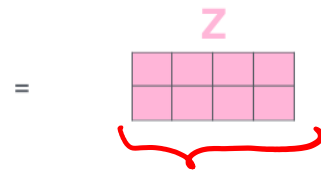


2) Multiply with a weight matrix W^O that was trained jointly with the model

X

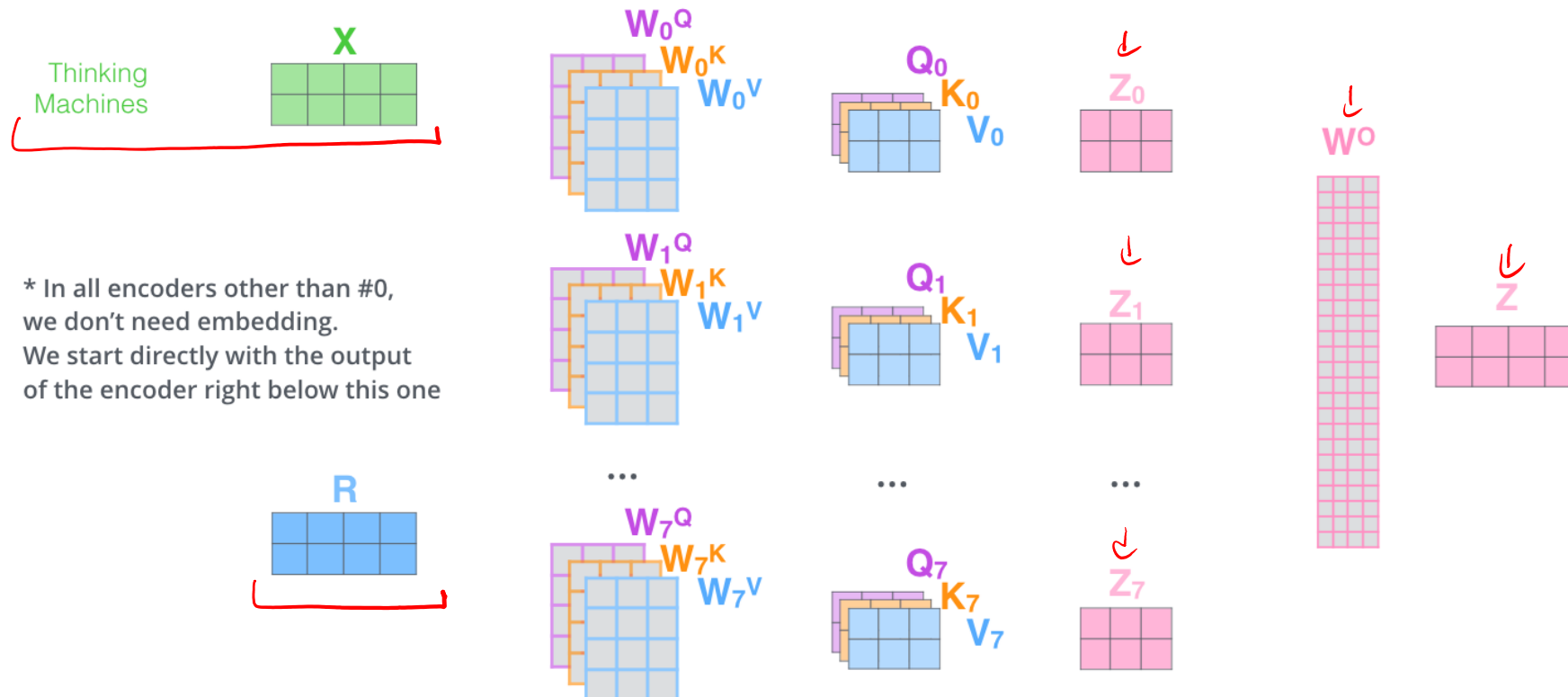


3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

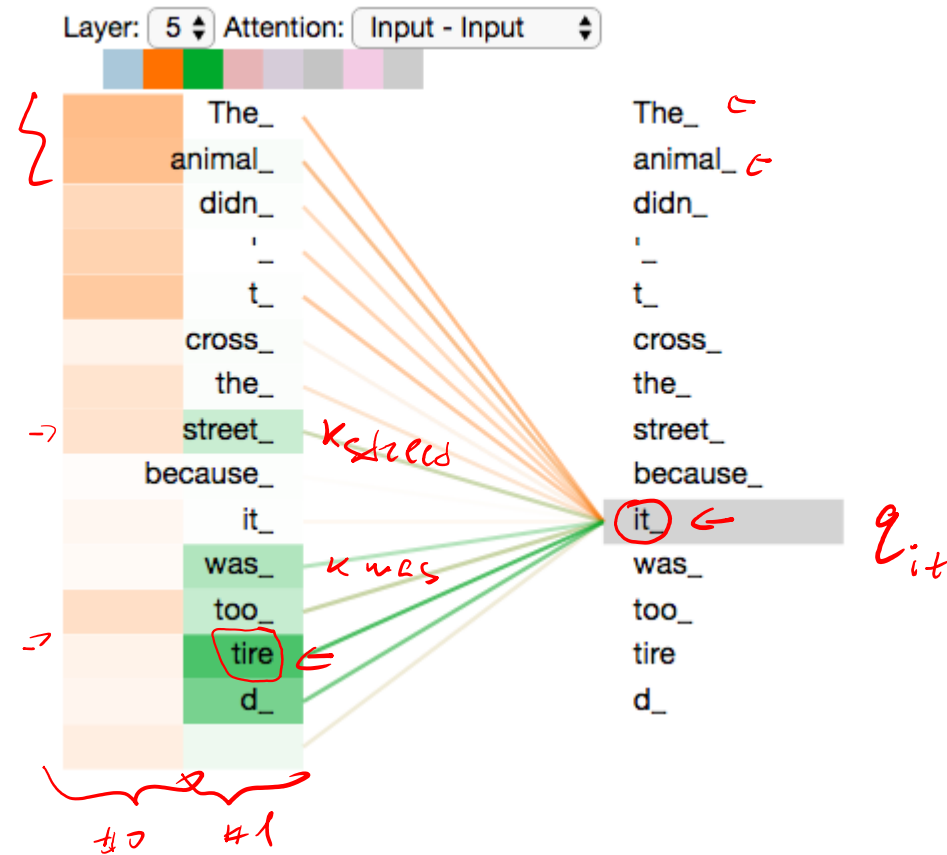


A multi-head attention overview

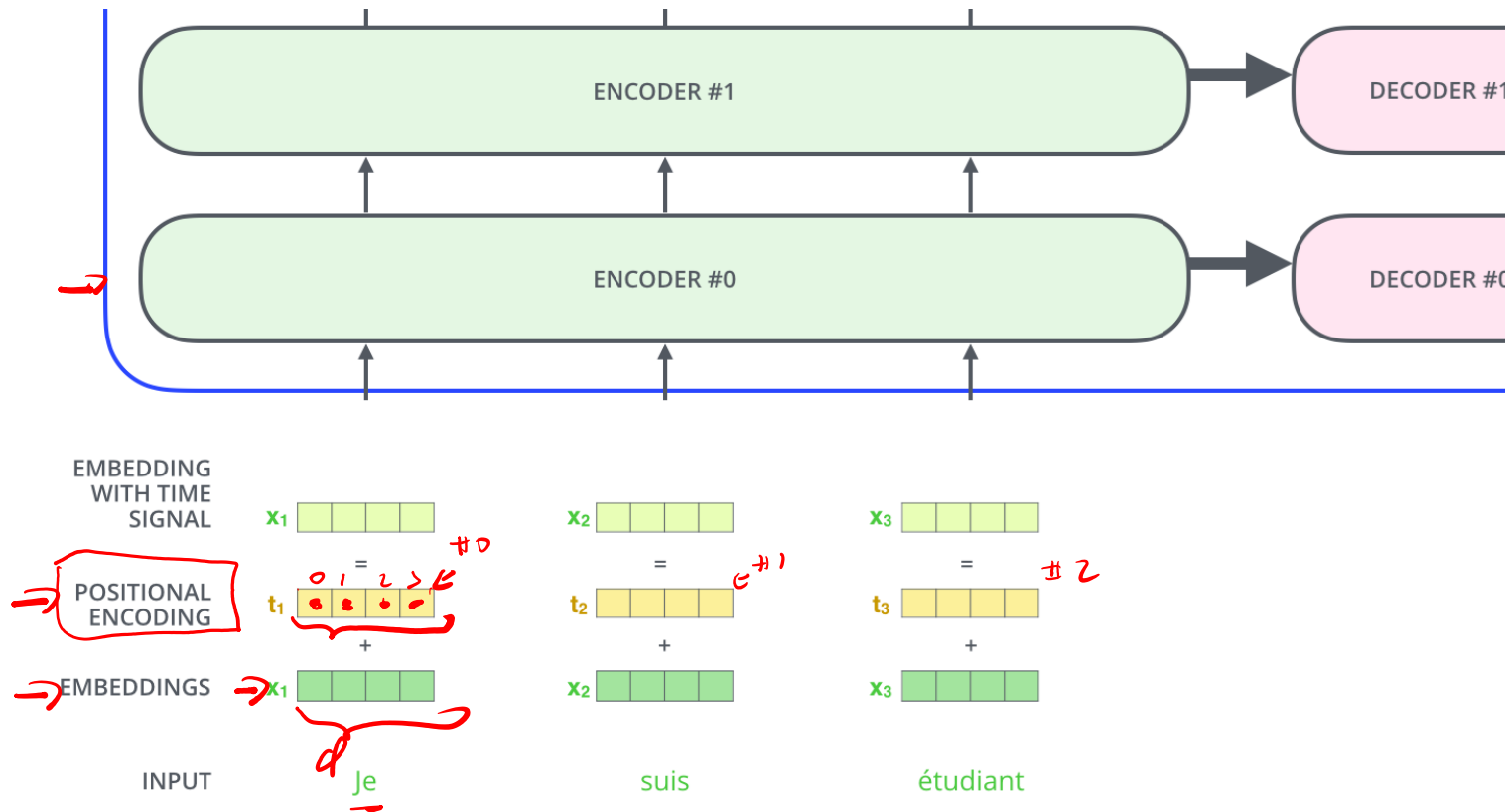
- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



Each head focuses on a specific representation



As opposed to RNNs, Transformers do not track the position implicitly

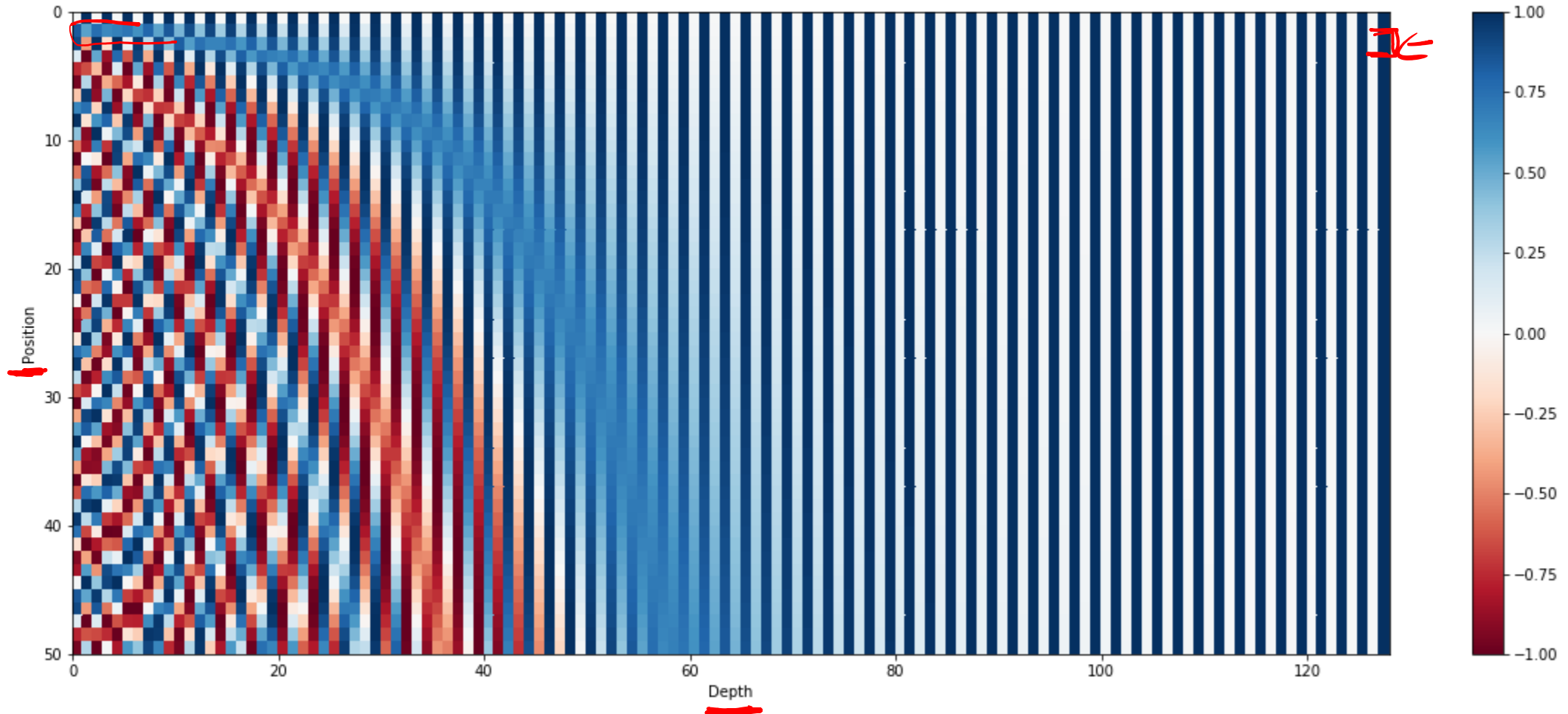


Absolute Positional Encoding

$$\rightarrow \text{PE}(\underbrace{pos}, \underbrace{2i}) = \underbrace{\sin}\left(\underbrace{pos/10000}_{\text{frequency}}^{\text{pos}} \underbrace{^{2i/d_{model}}}_{\text{dim}}\right)$$

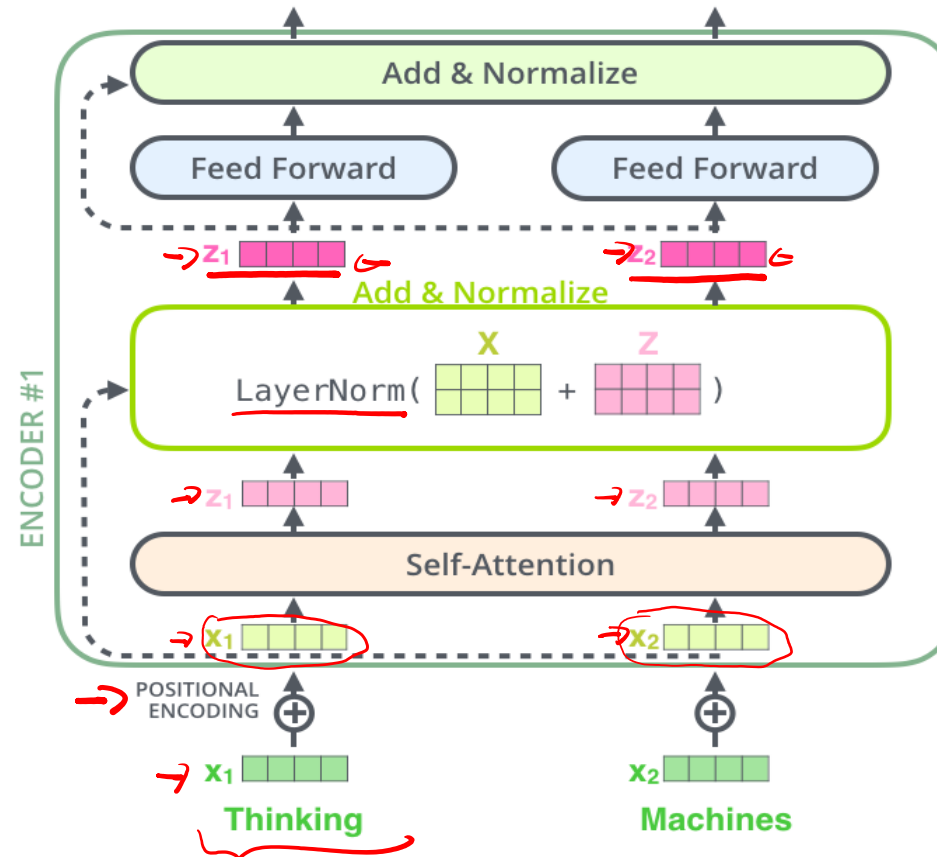
$$\rightarrow \text{PE}(\underbrace{pos}, \underbrace{2i + 1}) = \underbrace{\cos}\left(\underbrace{pos/10000}_{\text{frequency}}^{\text{pos}} \underbrace{^{2i/d_{model}}}_{\text{dim}}\right)$$

What it looks like

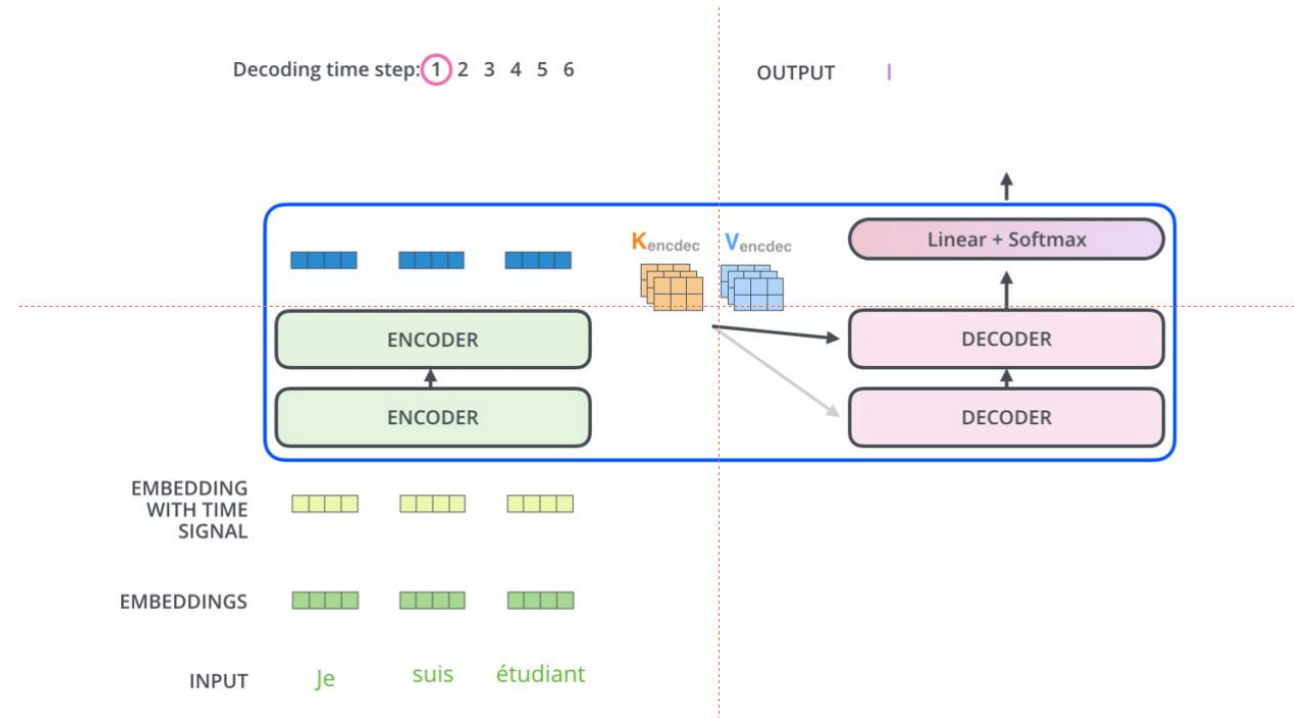


Source: https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

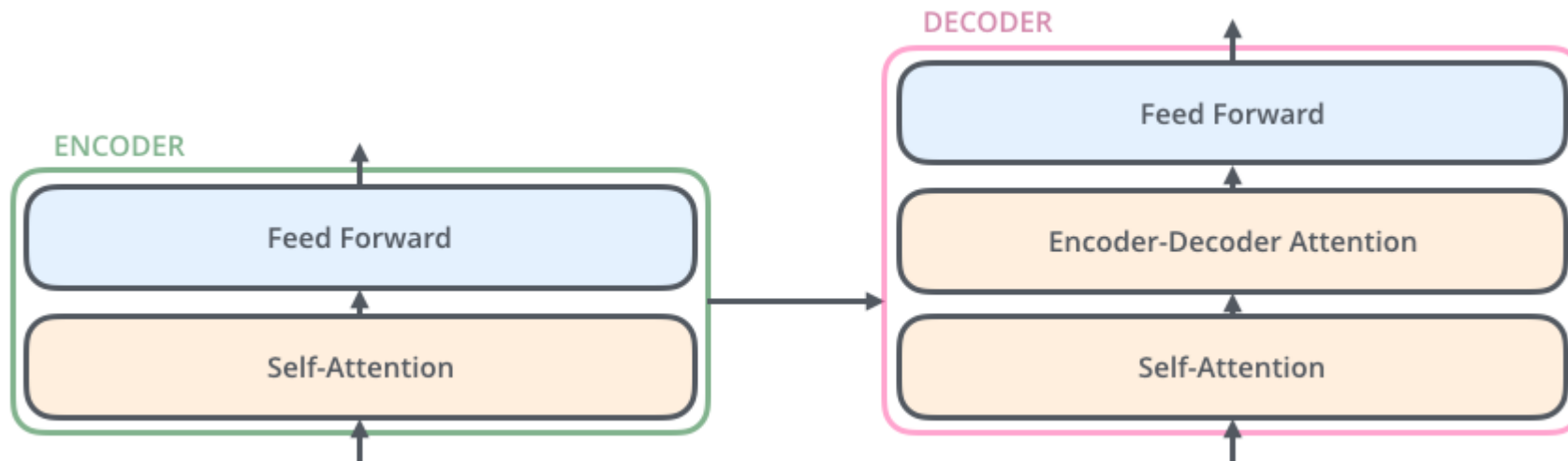
Skip Connection and Layer Normalization for a robust training



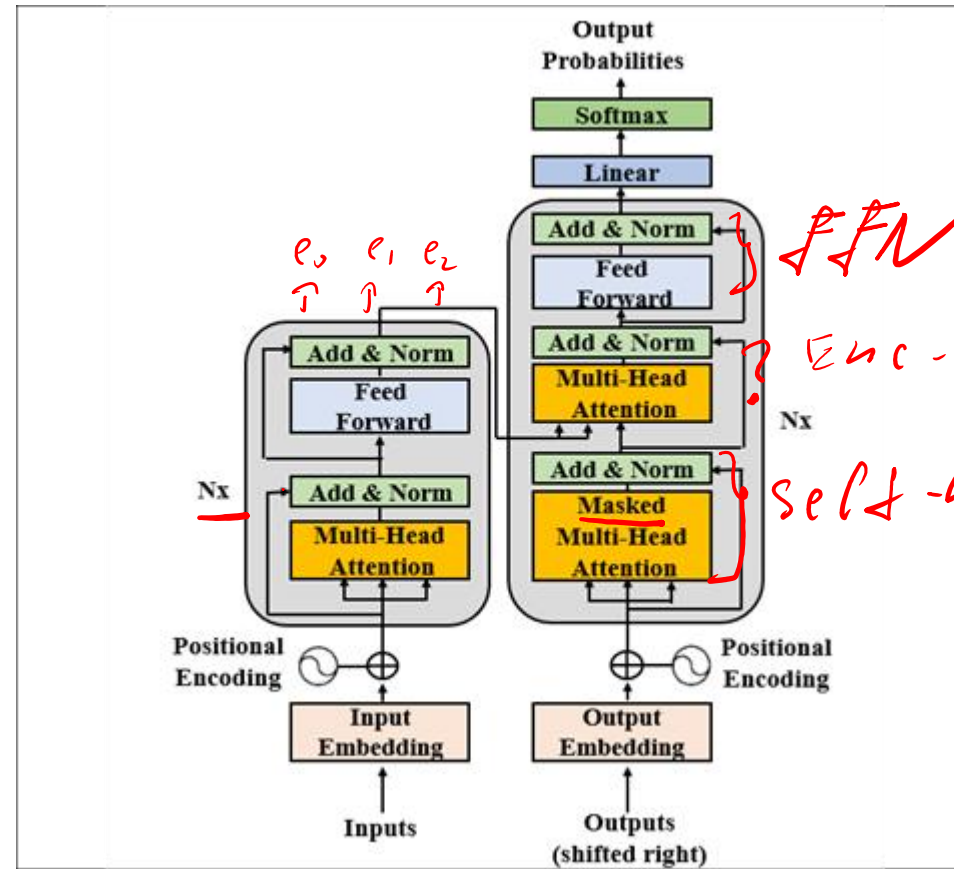
Bringing it all together



Always two there are. No more, no less. An Attention and an FFN. © Yoda



Attention is all you need



Attention is all you need (but not really)

